Feature	Description
ContentDialog	The ContentDialog control from Windows Phone 8.1 is now available for Universal Windows apps across all device families. ContentDialog lets you display a customizable modal dialog that works great across the full spectrum of devices.
Pivot	The Pivot control from Windows Phone 8.1 is now available for Universal Windows apps across all device families. You can now use the same Pivot control for in your app for mobile and desktop devices. Pivot provides adaptive behavior based on the screen size and input type. You can style a Pivot control to provide tab-like behavior, with different views of information in each pivot item.

Text

Feature	Description
Windows core text APIs	The new Windows.UI.Text.Core namespace features a client-server system that centralizes the processing of keyboard input into a single server. You can use it to manipulate the edit buffer of your custom text input control. The text input server ensures that the contents of your text input control and the contents of its own edit buffer are always in sync, via an asynchronous communication channel between the app and the server.
Vector icons	The Glyphs element has the new IsColorFontEnabled and ColorFontPalleteIndex properties to support color fonts; now you can use a font file to render font-based icons. When you use ColorFontPalleteIndex for color palette switching, a single icon can be rendered with different color sets; for example, to show an enabled and disabled version of the icon.
Input Method Editor window events	Users sometimes enter text through an Input Method Editor that shows in a window just below a text input box (typically for East Asian languages). You can use the CandidateWindowBoundsChanged event and DesiredCandidateWindowAlignment property on TextBox and RichEditBox to make your app UI work better with the IME window.
Text composition events	TextBox and RichEditBox have new events to inform your app when text is composed using an Input Method Editor: TextCompositionStarted, TextCompositionEnded, and TextCompositionChanged. You can handle these events to coordinate your app code with the IME text composition process. For example, you could implement inline auto-completion functionality for East Asian languages.

Feature	Description
Improved handling of bi-directional text	XAML text controls have new API to improve handling of bi-directional text, resulting in better text alignment and paragraph directionality across a variety of input languages. The default value of the TextReadingOrder property has been changed to DetectFromContent, so support for detecting reading order is enabled by default. The TextReadingOrder property has also been added to PasswordBox, RichEditBox, and TextBox. You can set the TextAlignment property on text controls to the new DetectFromContent value to opt-in to having alignment detected automatically from the content.
Text rendering	In Windows 10, text in XAML apps now renders, in most situations, at nearly twice the speed of Windows 8.1. In most cases, your apps will benefit from this improvement without any changes. In addition to faster rendering, these improvements also reduce typical memory consumption of XAML apps by 5%.

Application model

Feature	Description
Cortana	Extend the basic functionality of Cortana with voice commands that launch and execute a single action in an external application. By integrating the basic functionality of your app, and by providing a central entry point for the user to accomplish most of the tasks without opening your app directly, Cortana can act as a liaison between your app and the user. In many cases, this can save the user significant time and effort. Learn how to integrate your app into the Cortana canvas. If you need ideas you can refer to the design recommendations and UX guidelines specific to Cortana in Design basics for Universal Windows apps.
File Explorer	The new Windows.System.Launcher.LaunchFolderAsync methods let you launch File Explorer and displays the contents of a folder that you specify.
Shared storage	The new Windows.ApplicationModel.DataTransfer.SharedStorageAccessManager class and its methods let you share a file with another app by passing a sharing token when you launch the other app by using URI activation. The target app redeems the token to get the file shared by the source app.
Settings	Display built-in settings pages by using the ms-settings protocol with the LaunchUriAsync method. For example, the following code displays the page of Wi-Fi settings: **bool result = await Launcher.LaunchUriAsync(new Uri("ms-settings://network/wifi"));** For a list of the settings pages that you can display, see How to display built-in settings pages by using the ms-settings protocol.

Feature	Description
App-to-App communication	New app-to-app communication APIs in Windows 10 make it possible for Windows applications (as well as Windows Web applications) to launch each other and exchange data and files. Using these new APIs, complex tasks that would have required the user to use multiple applications can now be handled seamlessly. For example, your app could launch a social networking app to choose a contact, or launch a checkout application to complete a payment process.
App services	An app service is a way for an app to provide services to other apps in Windows 10. An app service takes the form of a background task. Foreground apps can call an app service in another app to perform tasks in the background. For reference information about the app service API, see Windows.ApplicationModel.AppService.
App package manifest	Updates to the package manifest schema reference for Windows 10 include elements that have been added, removed, and changed. See Element Hierarchy for reference info on all elements, attributes, and types in the schema.

Devices

Feature	Description
Microsoft Surface Hub	The Microsoft Surface Hub is a powerful team collaboration device and a large-screen platform for Universal Windows apps that run natively from Surface Hub or from your connected device. Build your own apps, designed specifically for your business, that take advantage of the large screen, touch and ink input, and extensive onboard hardware like cameras and sensors. Have a look at the design recommendations and UX guidelines specific to Surface Hub in Design basics for Universal Windows apps. These docs explain responsive design techniques for Universal Windows apps. For detail on supporting communal shared apps, see SharedModeSettings. For ink input and detail on support for multi-point inking on the new InkCanvas control, see Windows.UI.Input.Inking and Windows.UI.Input.Inking.Core. For handling sensor input, see Integrating devices, printers, and sensors.

Feature	Description
Location	Windows 10 introduces a new method to prompt the user for permission to access their location, RequestAccessAsync. The user sets the privacy of their location data with the **location privacy settings** in the **Settings** app. Your app can access the user's location only when: **Location for this device** is turned on *(not applicable for Windows 10 for phones)*, the location services setting "**Location**" is on, and under **Choose apps that can use your location**, your app is set to on. It's important to call **RequestAccessAsync** before accessing the user's location. At that time, your app must be in the foreground and **RequestAccessAsync** must be called from the UI thread. Until the user grants your app permission to their location, your app can't access location data.
AllJoyn	The Windows.Devices.AllJoyn Windows Runtime namespace introduces Microsoft's implementation of the AllJoyn open source software framework and services. These APIs make it possible for your Universal Windows device app to participate with other devices in AllJoyn-driven, Internet of Things (IoT) scenarios. For more details about the AllJoyn C APIs, download the documentation at The AllSeen Alliance. Use the AllJoynCodeGen tool included in this release to generate a Windows component that you can use to enable AllJoyn scenarios in your device app. **Note:** Windows 10 IoT Core is now available for a new class of small devices, allowing you to create "Internet of Things" (IoT) devices using Windows and Visual Studio. Learn more about Windows IoT on Windows Dev Center.
Printing APIs on mobile (XAML)	There is a single, unified set of APIs that let you print from your XAML-based UWP apps across device families, including mobile devices. You can now add printing to your mobile app by using familiar printing-related APIs from the Windows.Graphics.Printing and Windows.UI.Xaml.Printing namespaces.
Battery	The battery APIs in the Windows.Devices.Power namespace let your app learn more about any batteries that are connected to the device that's running your app. Create a Battery object to represent an individual battery controller or an aggregate of all battery controllers (when created by FromIdAsync or AggregateBattery, respectively). Use the GetReport method to return a BatteryReport object that indicates the charge, capacity, and status of the corresponding batteries.
MIDI devices	The new Windows.Devices.Midi namespace lets you create A=apps that can communicate with external MIDI devices, Apps and external devices that directly communicate with the Microsoft GS MIDI software synthesizer, and scenarios where multiple clients simultaneously access a single MIDI port.
Custom sensor support	The Windows.Devices.Sensors.Custom namespace allows hardware developers to define new custom sensor types, like a CO2 sensor.

Feature	Description
Host-based Card Emulation (HCE)	Host card emulation enables you to implement NFC card emulation services hosted in the OS and still be able to communicate with the external reader terminal via NFC radio. To trigger a background task to emulate a smartcard via NFC, use the SmartCardTrigger class. The EmulatorHostApplicationActivated value in the SmartCardTriggerType enum lets your app know that an HCE event has occurred.

Graphics

Feature	Description
DirectX	DirectX 12 in Windows 10 introduces the next version of Microsoft Direct3D, the 3D graphics API at the heart of DirectX. Direct3D 12 Graphics enables the efficiency and performance of a low-level, console-like API. Direct3D 12 is faster and more efficient than ever before. It enables richer scenes, more objects, more complex effects, and better use of modern graphics hardware.
SoftwareBitmapSource	In Universal Windows apps, you can use the new SoftwareBitmapSource type as a XAML image source. This lets you pass un-encoded images to the XAML framework to be immediately displayed on screen, bypassing image decoding by the XAML framework. You can achieve much faster image rendering, such as rendering low-lag photos directly from the camera, using custom image decoders, capturing frames from DirectX surfaces, or even creating in-memory images from scratch and rendering them all directly in XAML with low latency and low memory overhead.
Perspective Camera	In Universal Windows apps, XAML has a new Transform3D API that lets you apply perspective transforms to a XAML tree (or scene), which transforms all XAML child elements according to that single scene-wide transform (or camera). You could do this previously using MatrixTransform and complex math, but Transform3D greatly simplifies this effect, and also enables the effect to be animated. For more info, see the UIElement.Transform3D property, Transform3D, CompositeTransform3D, and PerspectiveTransform3D.

Media

Feature	Description

Feature	Description
HTTP Live Streaming	You can use the new AdaptiveMediaSource class to add adaptive video streaming capabilities to your apps. The object is initialized by pointing it to a streaming manifest file. Supported manifest formats include Http Live Streaming (HLS) and Dynamic Adaptive Streaming over HTTP (DASH). Once the object is bound to a XAML media element, adaptive playback begins. Properties of the stream, such as the available, minimum, and maximum bitrates, can be queried and set where appropriate.
Media Foundation Transcode Video Processor (XVP) support for Media Foundation Transforms (MFTs)	Windows apps that use Media Foundation Transforms (MFTs) can now use the **Media Foundation Transcode Video Processor** (XVP) to convert, scale, and transform raw video data: The new MF_XVP_CALLER_ALLOCATES_OUTPUT attribute enables the output to caller-allocated textures even in Microsoft DirectX Video Acceleration (DXVA) mode. The new IMFVideoProcessorControl2 interface lets your app enable hardware effects, query for supported hardware effects, and override the rotation operation performed by the video processor.
Transcoding	The new MediaProcessingTrigger API lets your app perform media transcoding in a background task, so your transcoding operations can continue even when your foreground app has been terminated.
MediaElement media failure events	In Universal Windows apps, the MediaElement will play content containing multiple streams even if there's an error decoding one of the streams, as long as the media content contains at least one valid stream. For example, if the video stream in a content containing an audio and a video stream fails, the MediaElement will still play the audio stream. The PartialMediaFailureDetected notifies you that one of the streams within a stream could not be decoded. It also lets you know what type of stream failed so that you can reflect that info in your UI. If all of the streams within a media stream fail, the MediaFailed event is raised.
Support for adaptive video streaming with MediaElement	MediaElement has the new SetPlaybackSource method to support adaptive video streaming. Use this method to set your media source to an AdaptiveMediaSource.
Casting with MediaElement and Image	The MediaElement and Image controls have the new GetAsCastingSource method. You can use this method to programmatically send content from any media or image element to a broader range of remote devices, like Miracast, Bluetooth, and DLNA. This functionality is enabled automatically when you set AreTransportControlsEnabled to true on a MediaElement.

Feature	Description
Media transport controls for desktop apps	The ISystemMediaTransportControls interface and related APIs allow desktop apps to interact with the built-in system media transport controls. This includes responding to user interactions with the transport control buttons, and updating the transport controls display to show metadata about currently playing media content.
Random-access JPEG encoding and decoding	New WIC methods IWICJpegFrameEncode and IWICJpegFrameDecode enable the encoding and decoding of JPEG images. You can also now enable indexing of the image data, which provides efficient random access to large images at the expense of a larger memory footprint.
Overlays for media compositions	The new MediaOverlay and MediaOverlayLayer APIs make it easy to add multiple layers of static or dynamic media content to a media composition. Opacity, position, and timing can be adjusted for each layer, and you can even implement your own custom compositor for input layers.
New effects framework	The Windows.Media.Effects namespace provides a simple and intuitive framework for adding effects to audio and video streams. The framework includes basic interfaces that you can implement to create custom audio and video effects and insert them into the media pipeline.

Networking

Feature	Description
Sockets	Socket updates include: **Socket broker:** The socket broker can establish and close socket connections on behalf of an app in any state of the app lifecycle. This makes apps and the services that they provide more discoverable. For example, by way of the socket broker, a Win32 service can still accept incoming socket connections even when it's not running. **Throughput improvements:** Socket throughput has been optimized for apps that use the Windows.Networking.Sockets namespace.
Background Transfer post-processing tasks	New APIs in the Windows.Networking.BackgroundTransfer namespace let you register groups of post-processing tasks. So your app can act on the success or failure of background transfers immediately, even if it's not in the foreground, instead of waiting for the next time the user resumes the app.
Bluetooth support for advertisements	With the Windows.Devices.Bluetooth.Advertisement namespace, your apps can send, receive, and filter Bluetooth LE advertisements.

Feature	Description
Wi-Fi Direct API update	The device broker is updated to enable pairing with devices without leaving the app. Additions to the Windows.Devices.WiFiDirect namespace also let a device make itself discoverable to other devices, and let it listen for incoming connection notifications. **Note:** In this release, the Wi-Fi Direct feature improvements are not built into the UX, and they support only push-button pairing. Also, this release supports only one active connection.
JSON support improvements	The Windows.Data.Json namespace now better supports existing standard definitions and the developer experience when converting JSON objects during debug sessions.

Security

Feature	Description
ECC encryption	New APIs in the Windows.Security.Cryptography namespace provide support for Elliptical Curve Cryptography (ECC), a public-key cryptography implementation based on elliptical curves over finite fields. ECC is mathematically more complex than RSA, provides smaller key sizes, reduces memory consumption, and improves performance. It offers Microsoft services and customers an alternative to RSA keys and NIST-approved curve parameters.
Microsoft Passport	Microsoft Passport is an alternative method of authentication that replaces passwords with asymmetric cryptography and a gesture. Classes in the Credentials namespace, such as KeyCredentialManger, make it easy for developers to create application using Microsoft Passport without the complexity of cryptography or biometrics.
Microsoft Passport for Work	Microsoft Passport for Work is an alternative method for signing in Windows using your Azure Active Directory account that does not use passwords, smart card, and Virtual Smart Cards. You can choose whether to disable or enable this policy setting.
Token Broker	Token Broker is a new authentication framework that makes it easier for apps to connect to online identity providers (like Facebook). Features such as account username and password management and a streamlined UI provide a greatly improved authentication experience for users.

System services

Feature	Description

Feature	Description
Power	Your Windows desktop application can now be notified when battery saver is engaged or disengaged. By responding to changing power conditions, your application has the opportunity to help extend battery life. GUID_POWER_SAVING_STATUS: Use this new GUID with the PowerSettingRegisterNotification function to be notified when battery saver is engaged or disengaged. SYSTEM_POWER_STATUS: This structure has been updated to support battery saver. The fourth member, *SystemStatusFlag* (previously named Reserved1, now indicates if battery saver is engaged or not. Use the GetSystemPowerStatus function to retrieve a pointer to this structure.
Version	You can use the Version Helper functions to determine the version of the operating system. For Windows 10, these helper functions include a new function, IsWindows10OrGreater. You should use the helper functions rather than the deprecated GetVersionEx and GetVersion functions when you want to determine the system version. For more information about how to get the system version, see Getting the System Version. If you do use the deprecated GetVersionEx or GetVersion function to get version information in an OSVERSIONINFOEX or OSVERSIONINFO structure, be aware that the version number that these structures contain increases from 6.3 for Windows 8.1 and Windows Server 2012 R2 to 10.0 for Windows 10. For more information about version numbers for the operating system, see Operating System Version. You also need to specifically target Windows 8.1 or Windows 10 in your application to get the correct version information for these versions with the GetVersionEx or GetVersion function. For information about how to target your application for these versions of Windows, see Targeting your application for Windows.
User information	New APIs in the Windows.System namespace make it easy to access information about a user, like their username and account picture. It also provides the ability to respond to user events such as log-in and log-out.
Memory management and profiling	Support for memory profiling API in Windows.System has been extended to all platforms, and their overall functionality has been enhanced with new classes and functions.

Storage

Feature	Description

Feature	Description
File-search APIs available for Windows Phone	As an app publisher, you can register your app to share a storage folder with other apps that you publish by adding extensions to the app manifest. Then call the Windows.Storage.ApplicationData.GetPublisherCacheFolder method to get the shared storage location. The strong security model of Windows Runtime apps typically prevents apps from sharing data among themselves. But it can be useful for apps from the same publisher to share files and settings on a per-user basis.

Tools

Feature	Description
Live Visual Tree in Visual Studio	Visual Studio has a new Live Visual Tree feature. You can use it while debugging to quickly understand the state of your app's visual tree, and discover how element properties were set. It also lets you change property values while your app is running, so you can tweak and experiment without having to re-launch.
Trace logging	TraceLogging is a new event-tracing API for user-mode apps and kernel-mode drivers; it builds on Event Tracing for Windows (ETW). This API provides a simplified way to instrument code and include structured data with events without requiring a separate instrumentation manifest XML file. WinRT, .NET, and C/C++ TraceLogging APIs are available to serve different developer audiences.

User Experience

Feature	Description
Speech recognition	Continuous speech recognition for long-form dictation scenarios is now supported by the Universal Windows Platform. See how to enable continuous dictation in the Speech interaction docs.

Feature	Description
Drag-and-drop capabilities between different application platforms	The new Windows.ApplicationModel.DataTransfer.DragDrop namespaces bring drag-and-drop functionality to Universal Windows apps. Previously, common drag-and-drop scenarios for desktop programs—such as dragging a document from a folder into an Outlook email message to attach it—are not possible with Universal Windows apps. Using these new APIs, your app can let users easily move data between different Universal Windows apps and the desktop. To support Drag and Drop between apps, these new APIs have been added to XAML: ListViewBase.DragItemsCompleted; UIElement: CanDrag, DragStarting, StartDragAsync, DropCompleted; DragOperationDeferral, DragUI, DragUIOverride; DragEventArgs: AcceptedOperation, DataView, DragUIOverride, GetDeferral, Modifiers; DragItemsCompletedEventArgs, DropCompletedEventArgs, DragStartingEventArgs
Custom window title bars	For UWP apps for the desktop device family, you can now use the ApplicationViewTitleBar class with the ApplicationView.TitleBar property and Window.SetTitleBar method to replace the default Windows title bar content with your own custom XAML content. Your XAML is treated as "system chrome", so Windows will handle the input events instead of your app. This means the user can still drag and resize the window, even when clicking on your custom title bar content.

Web

Feature	Description
Microsoft Edge	Microsoft Edge is the new default browser built for Windows 10. For more information and an overview of the developer features and standards included in Microsoft Edge, including the latest JavaScript features, see The Microsoft Edge Developer Guide.
WebView browsing	The WebView control uses the same rendering engine as the new Microsoft Edge browser. This provides the most accurate, standards-compliant mode of HTML rendering.
Off-thread WebView	You can specify a WebView.ExecutionMode to enable processing and display of web content on a separate background thread. This can improve performance in certain, specific scenarios.

Feature	Description
WebView.UnsupportedUriSchemeIdentified event	The new WebView.UnsupportedUriSchemeIdentified event lets you decide how your app should an unsupported URI scheme. You can handle this event to let your app provide custom handling of unsupported URI schemes. For the HTML WebView control, see the MSWebViewUnsupportedUriSchemeIdentified event.
WebView.NewWindowRequested event	The new WebView.NewWindowRequested event lets you respond when a script in a WebView requests a new browser window. For the HTML WebView control, see the MSWebViewNewWindowRequested event.
WebView.PermissionRequested event	The new WebView.PermissionRequested event lets WebView content leverage rich new HTML5 APIs that require special permission from the user, like geolocation. For the HTML WebView control, see the MSWebViewPermissionRequested event.
WebView.UnviewableContentIdentified event	The new WebView.UnviewableContentIdentified event lets you respond when the WebView is navigated to non-web content such as a PDF file or Office document. For the HTML WebView controls, see the MSWebViewUnviewableContentIdentified event.
WebView.AddWebAllowedObject method	You can call the new WebView.AddWebAllowedObject method to inject a WinRT object into a XAML WebView, and then call its functions from trusted JavaScript hosted in that WebView. For example, web content can show system notifications by requesting that its parent app call the ToastNotificationManager WinRT API. For the HTML WebView control, see the addWebAllowedObject method.
WebView.ClearTemporaryWebDataAsync method	When a user interacts with web content inside a XAML WebView, the WebView control caches data based on that user's session. You can call the new ClearTemporaryWebDataAsync method to clear this cache. For example, you can clear the cache when one user logs out of the app so another user can’t access any data from the previous session.

Experimental APIs

Article • 10/20/2022

Experimental APIs are in the early stages of design and are likely to change as the owners incorporate feedback and add support for additional scenarios.

These APIs are flighted externally using Windows Insider SDKs so that developers can try them out and provide feedback before they become part of the official platform. While they are flighted, there is no promise of stability or commitment.

Consuming experimental APIs

Intellisense will let you know if an API is experimental. You will also get a compiler warning when you use an experimental API such as "... is for evaluation purposes only and is subject to change or removal in future updates".

These warnings help protect you from creating dependencies on experimental APIs in production projects. For experimental projects, you can ignore or disable these warnings.

By default, these APIs are disabled at runtime and calling them will result in a runtime exception. This is another safeguard to help prevent inadvertent dependencies and broad distribution of apps that consume experimental APIs.

To enable these APIs for experimentation, use the Windows Device Portal (WDP) Features plug-in on the target device to enable the feature corresponding to the API you want to call.

Documentation for a particular experimental API is at the discretion of the team that owns it.

Providing feedback

If you've tried an experimental API and would like to provide feedback, use the **Developer Platform** category in the Windows Feedback Hub.

Reference for Windows Runtime API namespaces

Windows Runtime (WinRT) is a set of application programming interfaces (APIs) that allow developers to create applications for the Windows operating system, designed to be language-independent so that they can be used from various programming languages, including C++, C#, or JavaScript. WinRT APIs are based on the Component Object Model (COM) and provide a modern, object-oriented approach to Windows development. WinRT APIs enable access to a wide range of Windows features and services, such as file handling, networking, and user interface elements.

API reference

REFERENCE

Windows Runtime (WinRT) namespaces

Windows UI Library (WinUI) namespaces

.NET APIs

Advertising SDK namespaces

Store Services SDK namespaces

OVERVIEW

Extension SDKs and API contracts

Win32 and COM APIs

Schema reference

REFERENCE

File and XML schema specifications

App package manifest schema

Tile, toast, and badge schemas

App Installer file schema

Languages and toolkits

REFERENCE

C# reference

C++/WinRT reference (winrt namespace)

C++/CX reference

Microsoft Interface Definition Language 3.0